

AI技术应用场景匹配报告

1. 报告背景

本报告面向“旧的怎么办”校园二手网项目，在前期已经完成的《软件过程定义》基础上，结合 2024-2026 年 AI 在软件工程领域的最新应用趋势，对 AI 技术在本项目中的适配场景进行识别、筛选与匹配分析。报告目的不是泛泛讨论“AI 很重要”，而是要回答两个更具体的问题：第一，近三年 AI 在软件工程生命周期的哪些环节已经形成了较成熟、可落地的应用成果；第二，这些成果放到本项目已经定义的软件过程中，究竟在哪些活动上最能提升效率、质量和交付稳定性。

根据前期定义，本项目的软件过程已按 ISO/IEC 12207、15504、33001 的思想，划分为项目策划与控制、需求获取与分析、架构与详细设计、实现与集成、测试与验证、发布与交付、评审等过程。项目本身采用 4 人小团队、6 个 Sprint 的 Scrum 迭代方式，强调 MVP 优先、双端并行开发、阶段性交付和周例会复盘。因此，本报告对 AI 场景的选择也必须遵循“小团队可用、课程项目可落地、对当前流程有直接增益”三个原则。

2. 2024-2026 年 AI 在软件工程领域的技术趋势研究

2.1 需求分析阶段

2024-2026 年，AI 在需求分析阶段的应用已经从“聊天式需求整理”向“结构化需求工程辅助”升级，主要体现在三个方面。

第一，AI 辅助需求捕获已经较为成熟。大模型可以对访谈记录、讨论纪要、聊天记录、原始想法进行归纳，自动抽取用户角色、业务目标、约束条件和关键场景，减少需求人员手工整理的时间成本。很多团队已经把 AI 作为会议纪要整理器、用户故事生成器和需求补全助手来使用，使零散输入更快转化为规范的需求条目。国内研究如 [ChatModeler: 基于大语言模型的人机协作迭代式需求获取和建模方法](#) 就体现了“人机协作、迭代获取、逐步建模”的典型路线。

第二，需求文档自动生成能力明显增强。2025 年关于“利用大语言模型进行软件需求形式化”的综述研究指出，研究热点已从简单文本生成，扩展到把自然语言需求转化为更规范、可验证的规格表达，并支持不断迭代修正 [ACM Survey Draft on Formalising Software Requirements with Large Language Models](#)。这说明 AI 已不再只是“帮写文档”，而是在推动需求从非结构化走向半结构化甚至形式化。

第三，需求一致性验证正在成为新重点。一些新工具和研究开始利用 LLM + 检索、规则校验或知识图谱，发现需求中的歧义、冲突、遗漏和不一致问题。例如面向需求质量评估的 SRA 系统就突出强调需求清晰性、完整性和一致性的自动审查能力 [SRA](#)。此外，关于生成式 AI 在需求工程中的系统性综述也表明，需求分析、需求获取、需求规约和需求验证已成为 2024-2025 年研究增长最快的子方向 [Generative AI for Requirements Engineering: A Systematic Literature Review](#)。对软件工程而言，这意味着 AI 正逐步承担“需求质量守门员”的角色。

总体来看，需求阶段的 AI 趋势可以概括为：从“整理文本”走向“生成结构化需求”，再走向“自动检查需求质量与一致性”。

2.2 系统设计阶段

在系统设计阶段，2024-2026 年 AI 的作用主要体现为“方案建议 + 模型生成 + 设计校验”。

第一，AI 驱动的架构设计建议日趋实用。当前主流做法并不是完全让 AI 代替架构师，而是让 AI 根据项目规模、访问模式、技术栈和质量属性目标，给出候选架构方案、模块划分建议和关键权衡点。2026 年有关软件架构设计的实践总结明确提出，“人负责设计，AI 负责验证”逐渐成为更可靠的使用方式，AI 更擅长帮助团队暴露隐藏约束、发现容量风险和补齐质量属性问题 [AI in Software Architecture Design](#)。

第二，自动生成 UML、流程图和组件模型的能力大幅提升。2025-2026 年，多种工具已经支持根据自然语言需求、接口描述甚至现有代码自动生成类图、时序图、组件图或 C4 模型。相关研究如 NOMAD 提出以多智能体方式完成从自然语言需求到 UML 类图的生成，并在多个案例中优于基线方法 [NOMAD](#)。这表明“文本到模型”的设计支持正在成为现实能力，而不是简单演示。

第三，设计模式推荐与架构重构建议逐步普及。AI 可以根据业务场景、代码组织方式和非功能需求，建议是否采用 MVC、观察者、策略、工厂等模式，或者提示目前设计中存在过度耦合、职责混乱、模块边界不清的问题。对学生项目而言，这一能力尤其重要，因为团队成员架构经验有限，AI 可以成为“第二个架构评审人”。

因此，系统设计阶段的趋势是：AI 不再只生成“看起来像图”的文档，而是逐步参与从需求到结构模型、从结构模型到设计建议、从设计建议到风险验证的全过程。

2.3 编码实现阶段

编码实现阶段是 2024-2026 年 AI 落地最广泛、最成熟的领域。

第一，AI 代码生成和自动补全已经成为开发环境的标准能力。到 2025-2026 年，GitHub Copilot、Cursor、Claude Code、JetBrains AI 等工具已经覆盖函数级、模块级甚至跨文件修改级别的代码生成。相关总结指出，AI 已从早期的“单行补全”发展为可执行多步任务的“自主编码”模式，能够围绕一个功能需求进行计划、编码、调试和迭代 [Autonomous coding](#)。GitHub 官方在 2025-2026 年的资料中也已将 Copilot 描述为覆盖“构建、测试、评审、交付”的全流程助手，而不再只是补全插件 [GitHub Copilot tutorial](#)、[Copilot CLI is now generally available](#)。

第二，智能代码审查进入常态化。AI 不仅能检查语法和风格，还能发现空指针风险、权限遗漏、异常处理不足、SQL 注入风险、重复代码和不合理复杂度。2026 年的开发工具综述把 AI 代码审查列为最核心的开发者能力之一，强调其可嵌入 IDE、PR 流程和 CI/CD 管道中，提升评审速度并减少人工遗漏 [Top 12 AI Developer Tools in 2026](#)。GitHub 官方文档也把“功能检查、上下文验证、代码质量评估、依赖审查、自动化校验”作为 AI 生成代码评审的标准步骤 [Review AI-generated code](#)、[About GitHub Copilot code review](#)。

第三，重构与迁移辅助能力明显增强。AI 可以执行变量重命名、方法抽取、类拆分、同步转异步、接口调用替换、重复逻辑合并等任务。这类任务对课程项目非常有价值，因为迭代式开发常常导致代码先跑通、后优化，AI 正好适合在 Sprint 中后期帮助团队快速偿还技术债。

编码阶段的趋势已经非常明确：AI 正在从“代码助手”转向“开发协作成员”，尤其适合处理高重复、规则明确、跨文件关联明显的工作。

2.4 测试验证阶段

测试验证阶段是近三年 AI 增长最快的方向之一。

第一，AI 生成测试用例越来越实用。AI 可以依据需求描述、接口文档和现有代码，自动生成单元测试、接口测试和部分 UI 测试脚本，并补充边界值、异常流、非法输入等人工容易遗漏的场景。很多开发实践文章已将 AI 生成测试脚本视为提升覆盖率、降低回归测试成本的高价值场景 [How AI Boosts Developer Productivity and Code Quality](#)。同时，学术研究也在提升这项能力的质量，例如 [TestART](#) 强调通过“生成 + 修复 + 覆盖反馈”的协同演化提升单元测试生成的可通过率和覆盖率。

第二，智能自愈测试脚本成为测试自动化的重要方向。尤其在前端和 UI 自动化测试中，传统脚本常因页面元素定位变化而失效，而 AI 驱动的工具正在通过语义识别、页面结构理解和历史执行记录，自动修复部分定位器变化带来的失败，从而降低脚本维护成本。中国信通院发布的《[智能化软件工程技术和应用要求 第3部分：智能测试能力](#)》以及相关解读资料，已经把测试分析与设计、测试开发与执行、测试评估与修复、测试者辅助、工程化能力等维度纳入统一能力框架[中国信通院联合发布智能化软件测试能力标准](#)。这说明智能测试已经从单点工具尝试走向能力体系建设。

第三，预测性缺陷检测正在从研究走向工程实践。AI 不只是“测试后发现问题”，还可以基于历史提交、缺陷分布、复杂度和改动范围预测哪些模块更可能出错，从而帮助团队优先测试高风险区域。这类能力对于 Sprint 时间紧、测试资源有限的小团队特别有意义。Gtest 全球软件测试技术峰会 2025 的资料中，已经明确出现“单接口/多接口用例智能生成、接口执行错误智能分析、接口变更智能监控”等高能落地场景[Gtest 2025 智能测试专场资料](#)。同时，Testin 官方发布的 [Testin XAgent](#) 也把“测试需求分析、测试用例设计、自动化脚本生成、脚本执行、结果分析”串成了测试全流程智能化闭环。

测试阶段的总体趋势是：AI 正把测试工作从“事后验证”推向“事前预测 + 自动生成 + 动态修复”的组合模式。

2.5 运维监控阶段

在运维监控阶段，AI 的核心价值集中在异常检测、根因分析和资源优化上。

第一，AI 异常检测已经广泛用于日志、指标和调用链分析。相比传统阈值告警，AI 更适合处理波动明显、模式复杂的业务流量场景，能识别“看起来没超阈值但实际上不正常”的异常行为。2026 年关于 AI 改造软件工程流程的综述普遍将监控与可观测性列为 AI 重点发力环节[How AI Is Transforming Software Engineering](#)。

第二，根因自动分析的价值持续提升。AI 可以结合日志、异常栈、变更记录和部署记录，快速推测问题更可能来自数据库、缓存、接口超时、权限配置还是前端调用错误，减少排障时的人肉比对成本。对双端项目而言，这种能力尤其适合处理“前端报错、后端看似正常、缓存又有影响”的联动问题。

第三，智能扩容与性能优化开始进入更普遍的工程实践。AI 可以基于历史流量规律、峰值分布和资源使用情况，建议缓存策略、并发参数、实例规格或扩容时机，从而降低资源浪费并提升稳定性。对于本项目这类学生项目，即使没有复杂生产流量，也可把 AI 用于压测分析和 Redis/MySQL 参数优化建议。

从 2024 到 2026，运维阶段 AI 的变化很明显：从“辅助看日志”逐步发展为“辅助判因、辅助优化、辅助决策”。

3. 结合本项目软件过程的场景匹配分析

结合《软件过程定义》，本项目最适合引入 AI 的并不是所有环节，而是那些同时满足“工作量大、重复性高、容易遗漏、直接影响迭代交付质量”的关键活动。综合分析后，建议优先选择以下 5 个应用场景。

3.1 场景一：需求获取与分析阶段的 AI 需求整理与一致性校验

对应软件过程：[2.1 需求获取与分析](#)

本项目在 M0 阶段需要输出 PRD、原型和 UI 设计稿，而校园二手交易业务又包含用户注册、商品发布、搜索筛选、订单流转、评价体系、后台审核等多个角色与流程。对 4 人团队而言，需求最容易出现的问题不是“不会写”，而是需求描述不统一、验收标准不清楚、流程前后矛盾。

AI 在这一阶段的最优用法包括：把讨论记录整理为用户故事；自动提炼功能列表、前置条件和异常流；检查“商品状态”“订单状态”“审核状态”等是否前后一致；根据原始描述自动补充验收标准。例如，AI 可以把“学生想发布闲置商品并在有人下单后完成交易”扩展为标准故事格式，并补出未支付取消、商品下架、库存状态同步等边界情况。

对本项目的价值主要体现在三点：一是减少 PRD 编写时间；二是降低后续设计返工；三是让前后端对需求理解更一致。这个场景应列为高优先级，因为它直接影响 Sprint 2 之后所有工作的准确性。

3.2 场景二：架构与详细设计阶段的 AI 架构建议与 UML 自动生成

对应软件过程：2.2 架构与详细设计

本项目采用 Spring Boot + Vue + 微信小程序 + MySQL + Redis 的双端架构，既有典型的前后端分层问题，也有商品、订单、评价、审核、在线沟通等业务对象之间的关系设计问题。对于课程项目团队来说，设计阶段的难点不在于“画不出图”，而在于“设计图是否真正反映业务逻辑并支持后续实现”。

AI 在这一阶段可承担两个高价值任务：其一，根据需求自动生成用例图、类图、时序图、接口说明草稿，帮助团队快速形成设计基线；其二，对当前设计进行合理性检查，例如是否存在实体职责混乱、接口粒度过粗、缓存使用时机不明确、订单状态机不完整等问题。

这一场景非常适合本项目，因为项目成员分工明确但角色交叉较多，AI 自动生成和校验设计产物后，可显著降低前后端沟通成本，尤其能帮助梁有黎负责的数据结构设计与唐龙逸负责的后端业务设计更快对齐。

3.3 场景三：实现与集成阶段的 AI 代码生成、补全与智能代码审查

对应软件过程：2.3 实现与集成

这是本项目最能直接提升效率的环节。原因在于 Sprint 2 至 Sprint 4 是开发工作量最高的阶段，既要完成后端接口，也要同步推进 PC 页面和微信小程序页面，还要处理接口联调、权限校验、缓存接入、安全加固等问题。

AI 在实现阶段可重点用于以下工作：快速生成 CRUD 接口、DTO/VO、Mapper、前端页面模板、接口调用封装和表单校验代码；在联调前自动检查参数命名、状态码设计和异常处理是否统一；在提交 PR 前进行 AI 代码审查，提示安全漏洞、重复逻辑、潜在空指针、SQL 拼接风险和权限遗漏。

这一场景之所以必须优先落地，是因为它同时提升“速度”和“质量”。对于学生项目，最常见问题是赶进度时把能跑作为第一目标，导致后面测试和演示时暴露大量细节问题。若把 AI 引入编码和评审流程，就可以把一部分错误前移到提交之前发现，从而缩短联调周期。

3.4 场景四：测试与验证阶段的 AI 测试用例生成与高风险缺陷预测

对应软件过程：2.4 测试与验证

本项目的软件过程已明确要求在 Sprint 4 进行回归测试，在 Sprint 5 做全量系统测试。问题在于，小团队通常没有足够时间为每个模块都编写高质量测试用例，最终容易只测主流程，不测边界条件。

AI 在这一阶段可显著提升效率。首先，它可以根据需求说明和接口文档自动生成功能测试点，覆盖正常流、异常流和边界值。其次，它可以根据历史修改范围和模块复杂度，提醒团队优先测试订单状态变更、商品审核、缓存同步、权限控制等高风险模块。再次，对于前端页面和接口联调，AI 还能帮助构造异常输入和兼容性场景。

对本项目而言，这一场景的收益很直接：在不显著增加人工成本的前提下，提高测试覆盖率，并帮助团队把有限测试资源集中到最可能出问题的地方。考虑到最终答辩对系统稳定性要求较高，这一场景应列为高优先级。

3.5 场景五：发布与交付阶段的 AI 异常检测与根因辅助分析

对应软件过程：2.5 发布与交付

虽然本项目规模不大，但仍然需要部署到测试或生产环境，且要支撑中期演示、最终答辩和线上运行。此时问题不再只是“功能是否实现”，而是“系统在演示环境下是否稳定、故障能否快速定位”。

AI 在这一阶段最适合做的不是复杂的自动运维，而是轻量级的日志归类、异常模式识别和根因提示。举例来说，当出现“商品详情页请求超时”“下单后库存未及时更新”“小程序登录态失效”等问题时，AI 可辅助分析是 Redis 缓存、数据库事务、接口超时、权限配置还是前端状态管理造成的。对于课程项目，这种能力能显著缩短演示前后的排障时间。

相较前四个场景，这一场景的优先级略低，但在中期答辩和最终验收前价值会迅速上升，因此更适合作为后期增强能力部署。

4. 场景优先级排序与匹配结论

综合本项目的团队规模、工期压力和交付目标，建议 AI 场景按以下顺序推进：

1. 实现与集成阶段的 AI 代码生成、补全与代码审查
2. 测试与验证阶段的 AI 测试用例生成与缺陷预测
3. 需求获取与分析阶段的 AI 需求整理与一致性校验
4. 架构与详细设计阶段的 AI 架构建议与 UML 自动生成
5. 发布与交付阶段的 AI 异常检测与根因辅助分析

之所以这样排序，是因为本项目最紧张的资源是开发时间，其次是测试时间，再其次才是完整设计建模与运维建设。编码和测试阶段最消耗人力，也最容易直接影响里程碑交付，因此优先引入 AI 最能体现短期收益。需求和设计阶段虽然也很重要，但项目规模相对有限，完全重型化建设的投入产出比不如编码和测试高。运维阶段则更适合作为答辩前的稳定性保障措施。

5. 结论

从 2024 到 2026，AI 在软件工程中的应用已经从局部工具增强，逐步走向覆盖需求、设计、编码、测试、运维全过程的工程化能力体系。其发展趋势十分明确：需求阶段强调结构化与一致性校验，设计阶段强调模型生成与架构验证，编码阶段强调生成、补全、审查与重构，测试阶段强调自动生成、脚本自愈和缺陷预测，运维阶段强调异常检测、根因分析和资源优化。

结合“旧的怎么办”校园二手网项目已定义的软件过程可以看出，本项目最适合引入 AI 的关键环节并不是平均分布在整个生命周期中，而是集中在需求整理、设计建模、编码联调、测试验证和发布排障五个节点。其中，以“AI 代码生成与审查”和“AI 测试用例生成与缺陷预测”最具直接收益，能够最明显提升小团队在短周期迭代下的交付效率和质量稳定性。

因此，《AI技术应用场景匹配报告》的最终结论是：本项目完全适合采用“轻量、分阶段、以提效为核心”的 AI 引入策略，不追求一次性构建完整 AI 工具链，而是优先把 AI 放到最消耗人力、最容易出错、最影响验收结果的关键过程活动中，形成可见的效率提升与质量增益。这种做法既符合课程项目的现实约束，也符合当前 AI 赋能软件工程的实际发展方向。

6. 参考依据

1. 中国信通院等，《[智能化软件工程技术和应用要求 第3部分：智能测试能力](#)》，2024。
2. 中国信通院 AI4SE 工作组，《[AI4SE 行业现状调查报告（2024年度）](#)》，2025 发布。
3. Gtest 全球软件测试技术峰会 2025 智能测试专场资料，[面向智能测试的接口平台构建](#)。
4. GitHub 官方文档，[Review AI-generated code](#)、[About GitHub Copilot code review](#)、[GitHub Copilot tutorial](#)。

5. Testin 官方资料, [Testin XAgent: 五大能力开启测试新纪元](#)。
6. 靳东明等, 《[ChatModeler: 基于大语言模型的人机协作迭代式需求获取和建模方法](#)》, 《计算机研究与发展》, 2024。
7. Beg 等, 《[ACM Survey Draft on Formalising Software Requirements with Large Language Models](#)》, 2025。
8. Giannouris 等, 《[NOMAD: A Multi-Agent LLM System for UML Class Diagram Generation from Natural Language Requirements](#)》, 2025。
9. Gu 等, 《[TestART: Improving LLM-based Unit Test via Co-evolution of Automated Generation and Repair Iteration](#)》, 2024。
10. 行业材料与综述补充: [AI in Software Architecture Design](#)、[Autonomous coding](#)、[How AI Is Transforming Software Engineering](#)。